Predictive AI Evaluation Challenge: Cold-Start Probability Modeling with Adaptive Calibration

Pardis Miri
CS321M Predictive Evaluation Challenge
Stanford University
parism

May 21, 2026

Abstract

This report describes my submission to the CS321M Predictive AI Evaluation Challenge. The task is to predict the probability that a known AI subject answers a cold-start benchmark item correctly, given only subject metadata, item text, benchmark, and condition. I explored smoothed empirical priors, embedding heads, item-response factor models, factor/prior ensembles, adaptive labeling, and Modal-trained variants. The best hidden Codabench score was -0.61 negative log-loss under username `parism`. The strongest family was not the largest neural model; it was a conservative smoothed-prior router with adaptive calibration. Heavier factor models, larger encoders, and learned residual probability models consistently overfit hidden data.

1 Problem Formulation

The challenge asks for a function

$$f(s, i, b, c) = \Pr(Y = 1 \mid s, i, b, c),$$

where s is the AI subject, i is the item text, b is the benchmark, c is the test condition, and Y is binary correctness. The hidden evaluation is item cold-start: test items have no public responses, while subjects mostly appear in the public matrix. The primary metric is mean log-likelihood, reported by Codabench as negative log-loss where higher is better. AUC-ROC is secondary and mainly diagnostic.

The competition also provides an adaptive-labeling channel. A submission may include `labeling.py`, whose `acquisition_function(input)` scores candidate inputs. The platform reveals the top $K = 5$ labels per data category and passes them into `predict(input, labeled)`. This makes calibration as important as ranking: a useful submission must return well-calibrated probabilities, and it must use a tiny number of revealed labels without overfitting them.

2 Data and Validation

I used the public Hugging Face dataset `aims-foundations/measurement-db`. The repository code follows the starter-kit data-loading pattern: response parquet files are listed explicitly, registry tables are loaded separately, and trace tables are excluded. The joined runtime view contains

`benchmark`, `condition`, `subject_content`, `item_content`, `label`.

Only binary correctness labels in $\{0, 1\}$ are used. The active public dataset contains roughly 3.4M binary rows, about 900 subjects, 150k unique items, 16 benchmarks, and 223 benchmark-condition cells.

For local validation I used whole-benchmark and benchmark-aware holdouts rather than random row splits. Random row validation leaks item and benchmark distributional structure and gives overly optimistic

estimates. I also built an offline adaptive-label simulator, `simulate_adaptive_labels.py`, which mimics the Codabench loop by selecting the top 5 acquired labels per benchmark-condition category and then scoring the remaining rows with the labeled context.

3 Methods

Smoothed-prior baseline. The simplest useful model estimates empirical correctness rates at several granularities and shrinks each cell toward a parent mean. The final prior uses the hierarchy

subject-benchmark-condition → subject-benchmark → benchmark-condition → subject/benchmark → global.

The first baseline averaged several cells. A stronger v2 baseline preferred the deepest populated cell, especially subject-benchmark-condition, with Bayesian shrinkage toward coarser cells. This improved local cold-start NLL substantially and became the backbone of the best submissions.

Embedding head. I trained a content-based head using SentenceTransformer item embeddings plus smoothed subject priors. The runtime artifact stores plain NumPy weights so that the submission does not require scikit-learn at inference. This approach underperformed the smoothed prior on Codabench, suggesting that item topic embeddings alone did not generalize well enough to hidden item difficulty.

Factor/PGE model. I implemented a three-stage prediction-guided-evaluation pipeline: first fit a rank-1 item-response/factor model with subject ability θ_s and item parameters (a_i, b_i); then train an MLP from MiniLM item embeddings to item parameters; finally predict

$$\sigma((a_i\theta_s - b_i)/T)$$

for cold-start items. Temperature T was fit on a cold-start holdout. Higher-rank factor models, larger encoders, and multi-seed variants reduced some local losses but did not improve hidden performance.

Robust prior router. The best family was a robust prior-only router between the older v1 weighted prior and the deeper v2 prior. Without labels, it uses a conservative default blend. With revealed labels, it compares the v1 and v2 priors on the labeled examples for the current benchmark-condition category and softly shifts the logit-space blend toward the better prior. A separate clipped logit blend shrinks predictions toward the observed mean of the revealed labels for that category.

Adaptive acquisition. The acquisition function must never fail: any exception or non-finite score triggers random fallback for the entire round. I therefore used simple deterministic scoring based on prior uncertainty and stable hashing. Attempts at hash-only, diversity-tiered, v1/v2 disagreement, and stronger label-correction acquisition did not improve hidden performance.

4 Experiments and Ablations

Table 1 summarizes the most important Codabench results. The best score was -0.61, reached by several conservative prior-family submissions. Larger or more aggressive variants generally worsened hidden log-loss.

The main ablation pattern was consistent. First, calibration and shrinkage mattered more than model complexity. The factor model had useful discriminative signal on some benchmarks, but it was poorly calibrated on others and was often dominated by smoothed priors. Second, adaptive labels helped only when used gently. Removing the label blend hurt, but stronger correction and smaller clips could also hurt. Third, local simulation was useful for screening variants, but it was not a perfect proxy: increasing the default v2 prior weight from 0.45 to 0.60 improved several public-data simulator seeds, yet dropped to -0.63 on hidden Codabench.

Submission	NLL	AUC	Interpretation
`robust_prior_v1`	−0.61	–	Conservative v1-heavy prior; tied best.
`robust_prior_v2`	−0.61	–	Deeper v2 prior blend; tied best.
`robust_direct_residual_w15`	−0.61	–	Tiny learned residual blend; tied best but did not beat prior.
`robust_prior_clip`	−0.62	–	Smaller label correction hurt.
`robust_hash`	−0.62	–	Hash-only acquisition worse than uncertainty.
`robust_prior_v2_w60`	−0.63	–	Higher v2 weight looked good locally but overfit hidden data.
`robust_direct_residual`	−0.64	–	Learned residual overfit when trusted more.
`robust_mpnet`	−0.65	–	Larger encoder/factor model did not generalize.

Table 1: Selected Codabench results. Scores are reported as negative log-loss, so less negative is better. The leaderboard screenshot for username `parism` shows best score −0.61 and AUC 0.68.

5 Failure Modes and Lessons

The strongest failure mode was hidden-distribution mismatch. Public-data validation rewarded deeper subject-benchmark-condition priors and higher v2 prior weights, while hidden data punished over-reliance on that direction. This suggests that hidden categories contain distribution shifts where public fine-grained cells are not always the right anchor.

The second failure mode was learned residual overfitting. A Modal-trained direct residual MLP had very strong calibration performance on a held-out public slice, but hidden performance worsened as soon as the residual received substantial weight. A 15% blend tied the best score; 5%, 35%, and 55% were worse. This indicates that the residual contains some signal but is not reliable enough to carry the prediction.

The final lesson is that this challenge rewards robust, boring probability estimation. The winning path for my submission was not a larger encoder or more expressive model, but a conservative prior hierarchy, defensive runtime code, stable adaptive acquisition, and clipped label-based calibration.

6 Reproducibility

The code is available at:

`https://github.com/paris007/torch_measure/tree/predictive-eval-challenge`.

The final Codabench-ready directories live under `codabench_submissions/`; `make_submission.py` builds flat ZIP archives with `model.py` at the archive root. The best submitted artifact for archival purposes is `dist/robust_prior_v1_submission.zip`; the best leaderboard entry is username `parism`, score −0.61, submission ID 743892.

References

[1] CS321M course staff. Predictive Evaluation Competition Handbook. Stanford University, 2026.

[2] Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Erlbaum, 1980.

[3] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *EMNLP*, 2019.